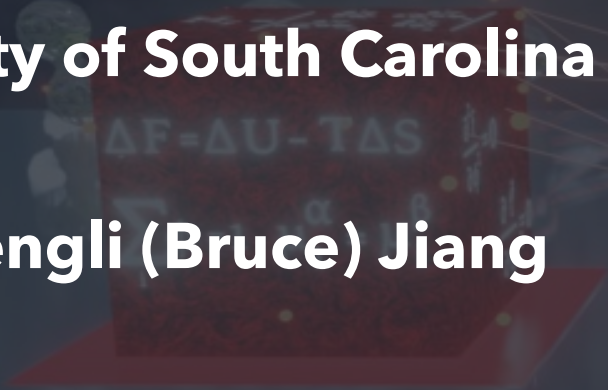


Hyperparameter Tuning

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



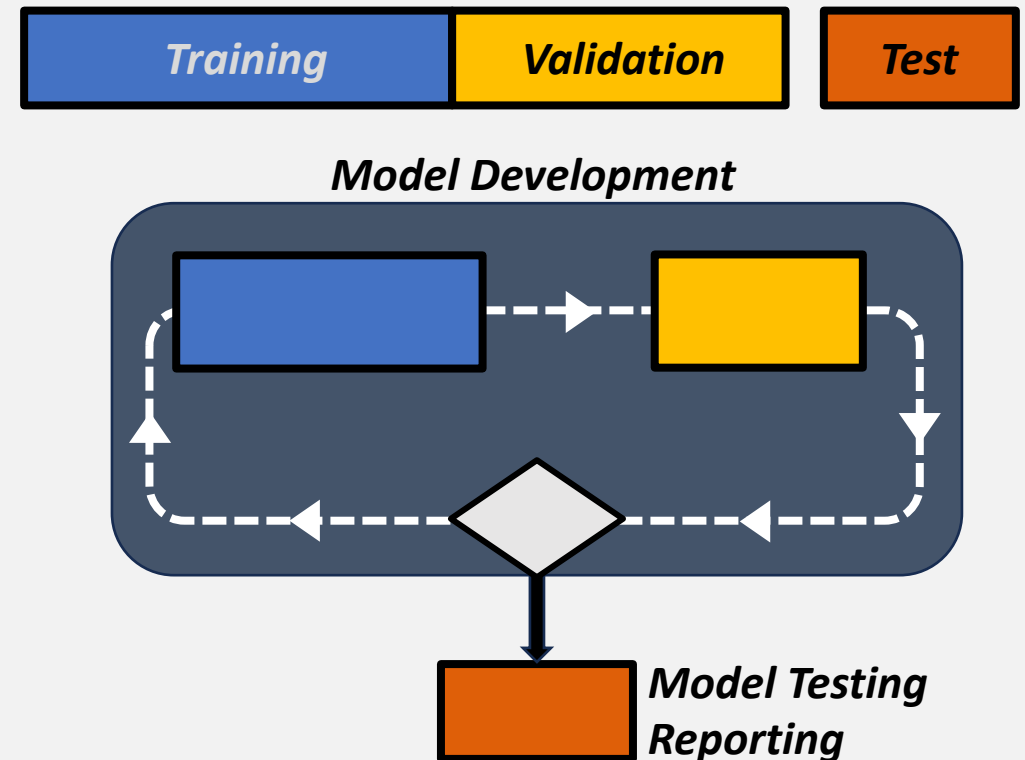
Training, Validation, and Test Sets Have Different Jobs

A primary goal of many machine learning regression problems is to achieve models that permit **generalization** (evaluation outside the domain of our dataset)

Cross-validation and/or the use of **train/validation/test splits** are two common strategies to mitigate overfitting a model to example data/inhibiting generalization

Basic premise of train-validation-test split

1. Shuffle dataset *randomly*
2. *Partition* your data into three groups, which we refer to as a **training** set, **validation** set, and a **test** set
3. How do we use these splits?
 - i. **training** – model is allowed to see this data and adjust its **parameters** accordingly
 - ii. **validation** – model does not adjust its **parameters** in response to this data but evaluation on the data provides feedback on generalizability for the given model type and mode of optimization
 - iii. **test** – model neither adjusts its parameters, model type, or mode of optimization – this is data that allows for critical assessment



The Best Data Split Depends on the Problem

A primary goal of many machine learning regression problems is to achieve models that permit **generalization** (evaluation outside the domain of our dataset)

Cross-validation and/or the use of **train/validation/test splits** are two common strategies to mitigate overfitting a model to example data/inhibiting generalization

Basic premise of train-validation-test split

1. Shuffle dataset *randomly*
2. *Partition* your data into three groups, which we refer to as a **training** set, **validation** set, and a **test** set
3. How do we use these splits?
 - i. **training** – model is allowed to see this data and adjust its **parameters** accordingly
 - ii. **validation** – model does not adjust its **parameters** in response to this data but evaluation on the data provides feedback on generalizability for the given model type and mode of optimization
 - iii. **test** – model neither adjusts its parameters, model type, or mode of optimization – this is data that allows for critical assessment



What is the right ratio of train : validation : test?

There is no single answer. Keep in mind...

- **how much data do you have?**
if data is precious, you may need to lean on more training data-without it, the model may just be garbage
- **how many hyperparameters do you have?**
you may need more on the validation side to practically guide model selection
- **what are you using your model for?**
either large or small test might be appropriate here.

Common Starting Points for a Three-Way Split

A primary goal of many machine learning regression problems is to achieve models that permit **generalization** (evaluation outside the domain of our dataset)

Cross-validation and/or the use of **train/validation/test splits** are two common strategies to mitigate overfitting a model to example data/inhibiting generalization

Basic premise of train-validation-test split

1. Shuffle dataset *randomly*
2. *Partition* your data into three groups, which we refer to as a **training** set, **validation** set, and a **test** set
3. How do we use these splits?
 - i. **training** – model is allowed to see this data and adjust its **parameters** accordingly
 - ii. **validation** – model does not adjust its **parameters** in response to this data but evaluation on the data provides feedback on generalizability for the given model type and mode of optimization
 - iii. **test** – model neither adjusts its parameters, model type, or mode of optimization – this is data that allows for critical assessment



What is the right ratio of train : validation : test?

As a rough guide, some standards are usually

- 50-80% for **training** data
- 10-30% for **validation** data
- 10-30% for **test** data

You will thus see

- We performed an 80/10/10 split
- ... 70/20/10
- ... 60/20/20
- etc.

Predicting Battery Lifetime from Early Measurements

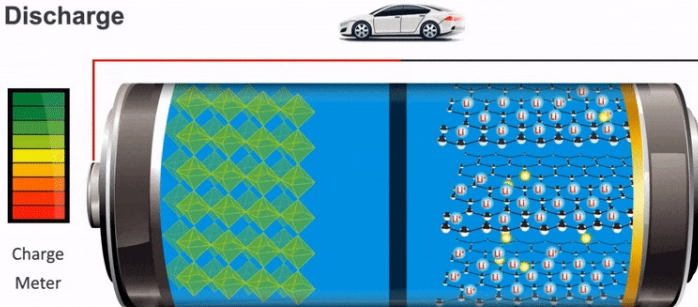
- Testing a battery until it reaches the end-of-life capacity threshold can take weeks.
- By the time we know which design last longest, we may already need to choose a material, a manufacturing process, or a charging strategy.

Suppose we have measured a battery for its first 100 charge-discharge cycles. Its capacity still looks healthy.

Can we use those early measurements to predict whether it will last 500 cycles or 1500 cycles?

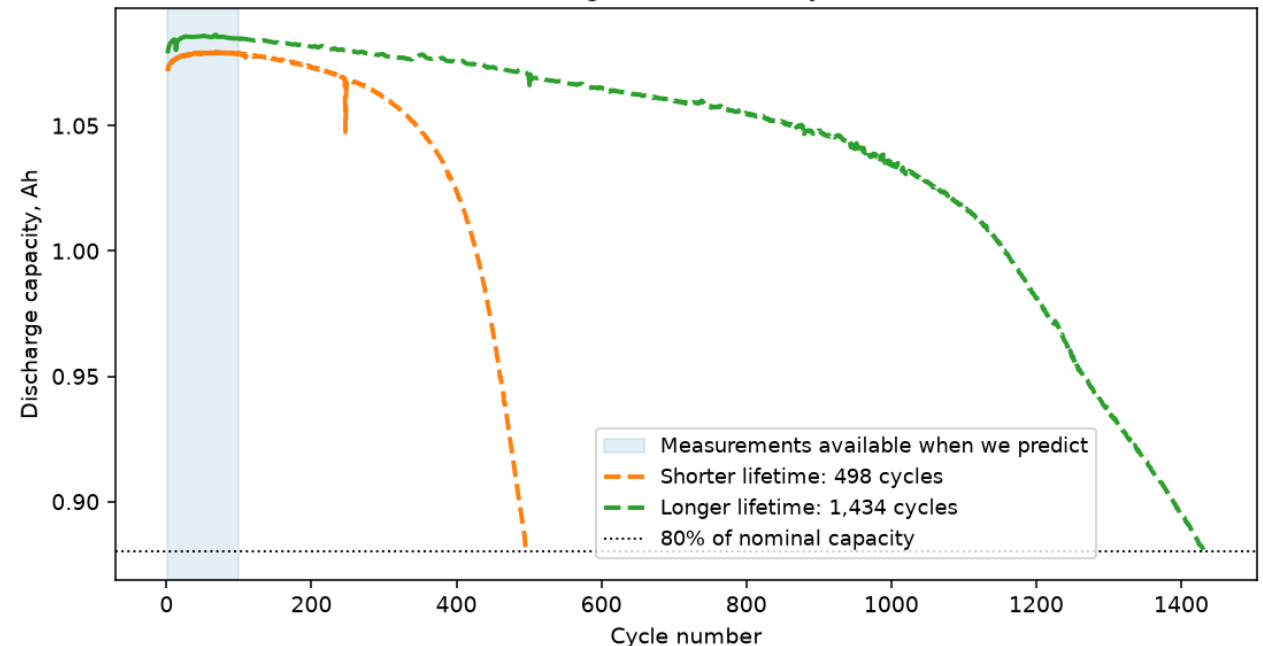
How Lithium-ion Batteries Work

Discharge



U.S. DEPARTMENT OF
ENERGY Office of ENERGY EFFICIENCY
& RENEWABLE ENERGY

How Long Will This Battery Last?



Predicting Battery Lifetime from Early Measurements

- Testing a battery until it reaches the end-of-life capacity threshold can take weeks.
- By the time we know which design last longest, we may already need to choose a material, a manufacturing process, or a charging strategy.

Suppose we have measured a battery for its first 100 charge-discharge cycles. Its capacity still looks healthy.

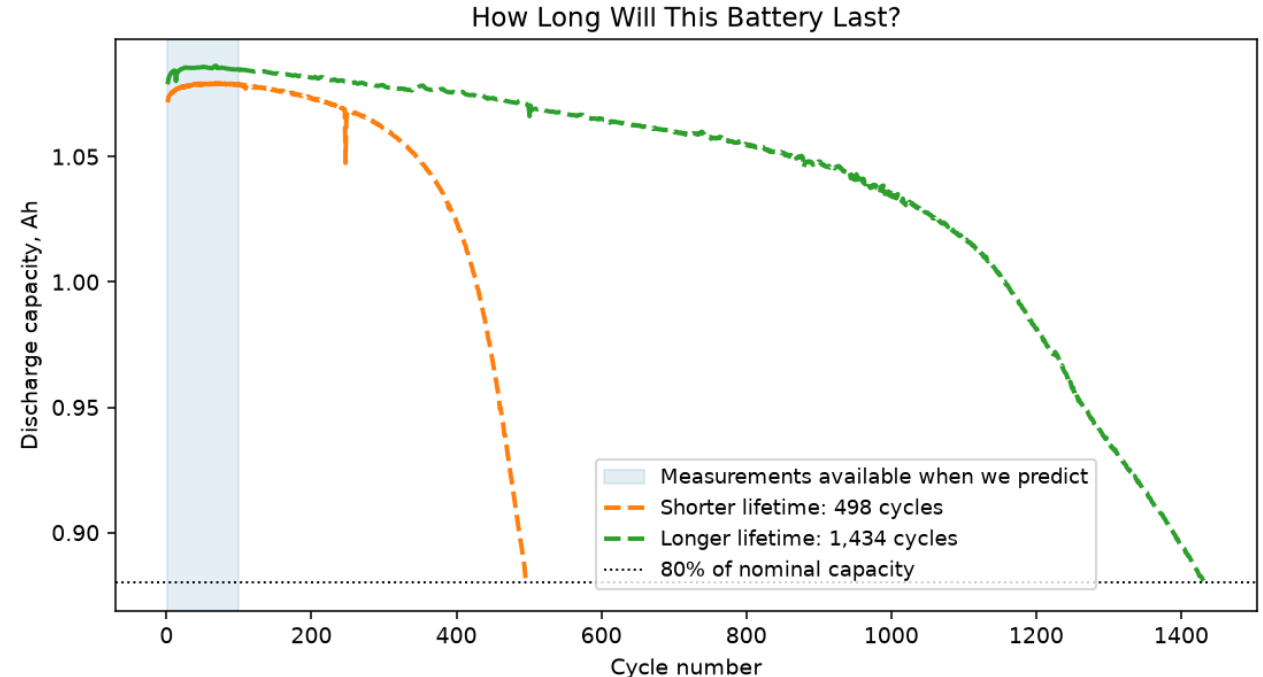
Can we use those early measurements to predict whether it will last 500 cycles or 1500 cycles?

Severson et al. studied **124 commercial lithium-ion batteries** with lithium iron phosphate/graphite chemistry under different fast-charging conditions. Their lifetimes ranged from approximately **150 to 2300 cycles**.

[10.1038/s41560-019-0356-8](https://doi.org/10.1038/s41560-019-0356-8)

<https://data.matr.io/1/>

Severson, K. A., Attia, P. M., et al. (2019). *Data-driven prediction of battery cycle life before capacity degradation*. Nature Energy, 4, 383–391.

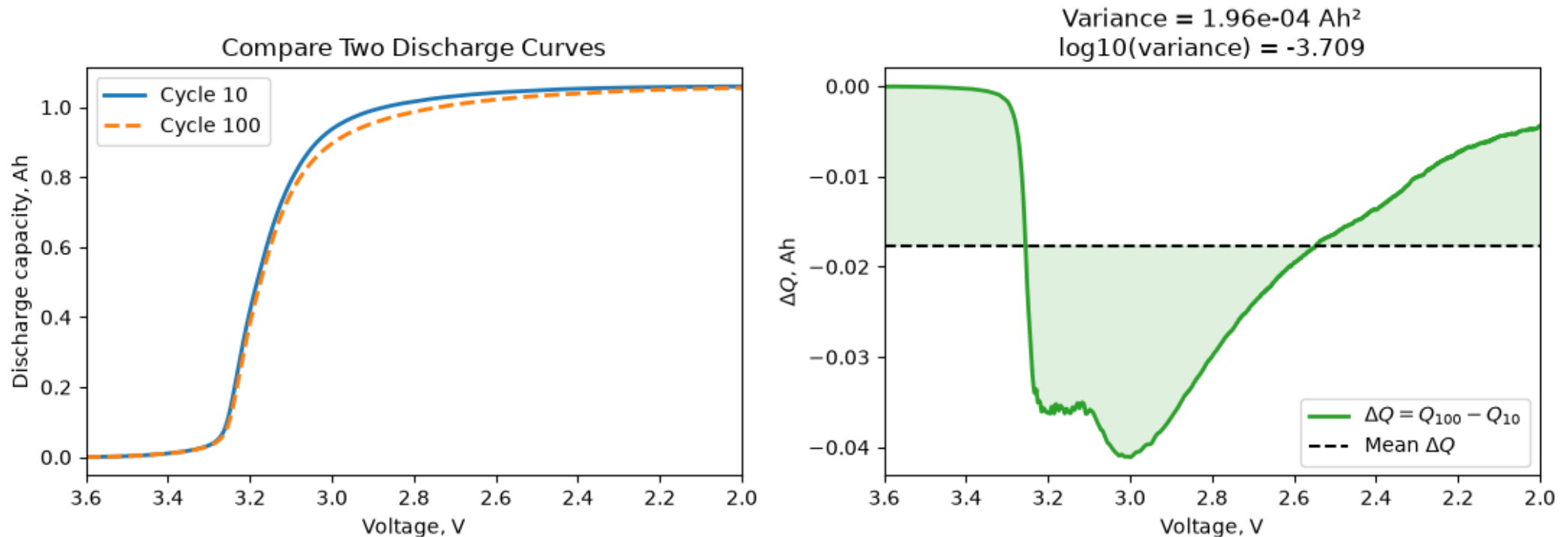


Features

$$\Delta Q(V) = Q_{100}(V) - Q_{10}(V)$$

Where $Q_n(V)$ is the discharge capacity at voltage V during cycle n . Each curve contains 1000 voltage points.

Features	Meaning
log10_var_dQ100_10	$\log_{10}[\text{Var}(\Delta Q)]$. Measures how unevenly the discharge curve changes across voltage. This is our starting feature.

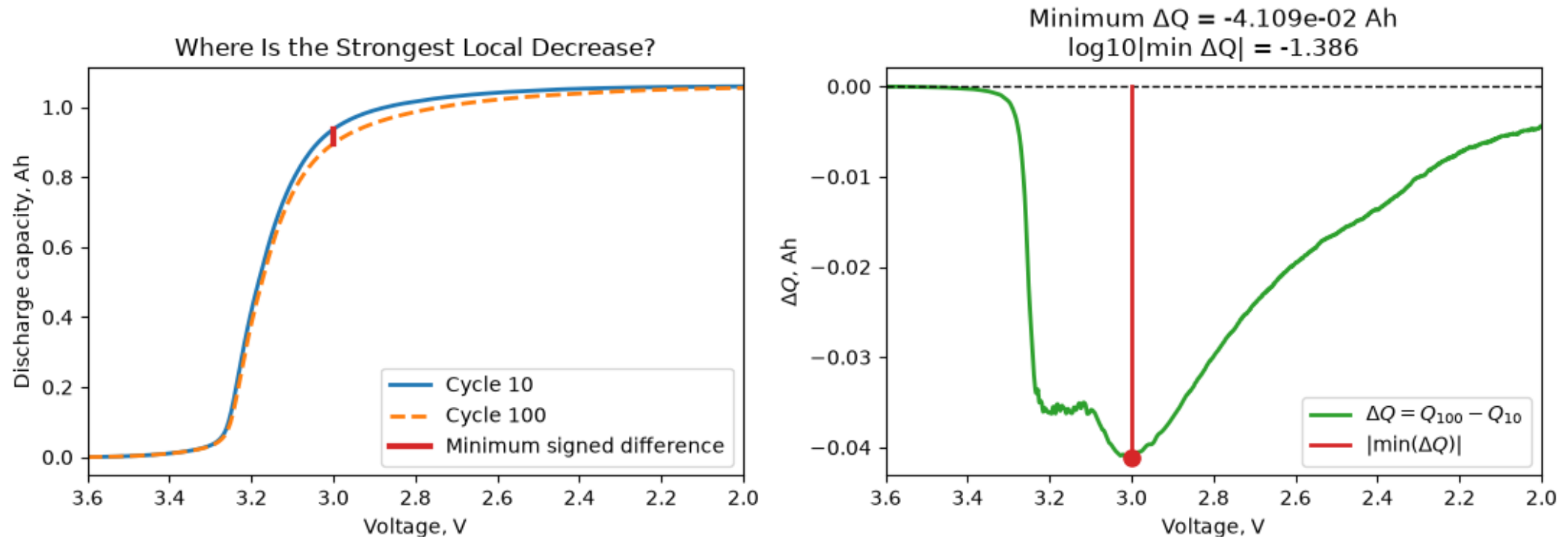


Features

$$\Delta Q(V) = Q_{100}(V) - Q_{10}(V)$$

Where $Q_n(V)$ is the discharge capacity at voltage V during cycle n . Each curve contains 1000 voltage points.

Features	Meaning
log10_abs_min_dQ100_10	$\log_{10}[\min(\Delta Q)]$. Takes the smallest signed difference, then its absolute value. When the minimum is negative, this measures the strongest local decrease.

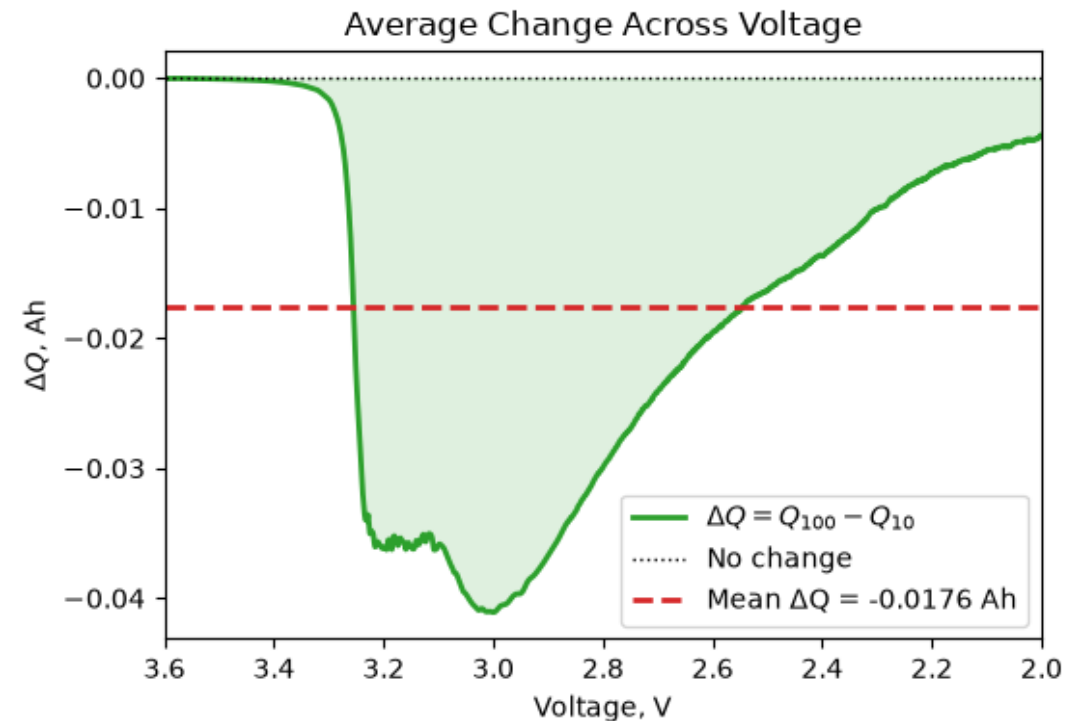
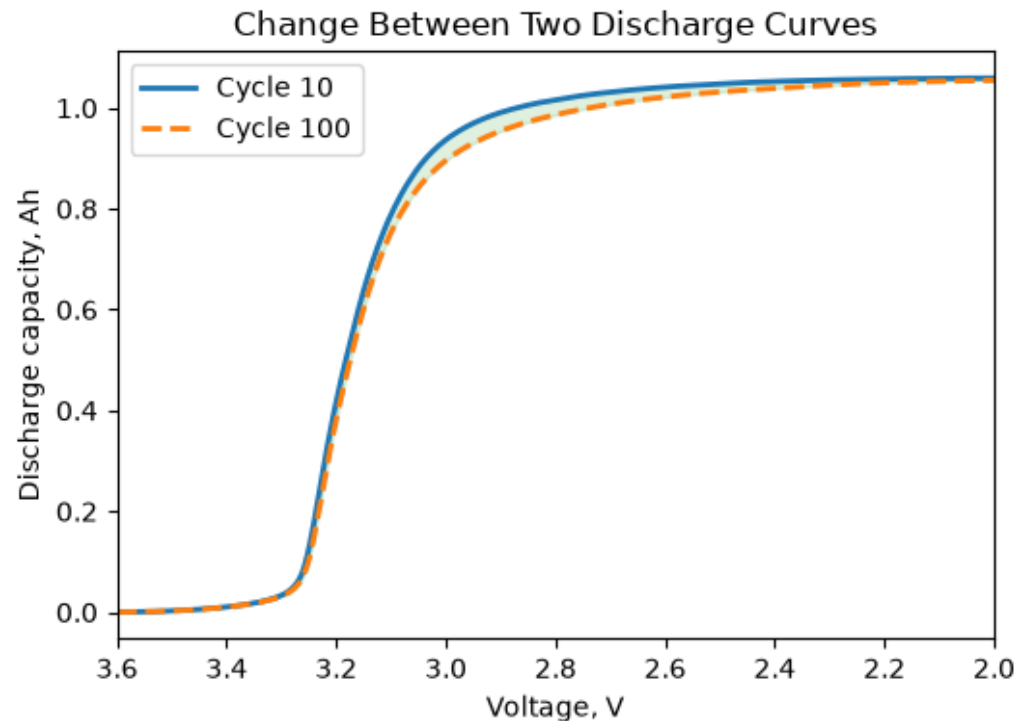


Features

$$\Delta Q(V) = Q_{100}(V) - Q_{10}(V)$$

Where $Q_n(V)$ is the discharge capacity at voltage V during cycle n . Each curve contains 1000 voltage points.

Features	Meaning
mean_dQ100_10_Ah	Mean of ΔQ across voltage points, in Ah. A negative value indicates an average decrease between cycles 10 and 100.

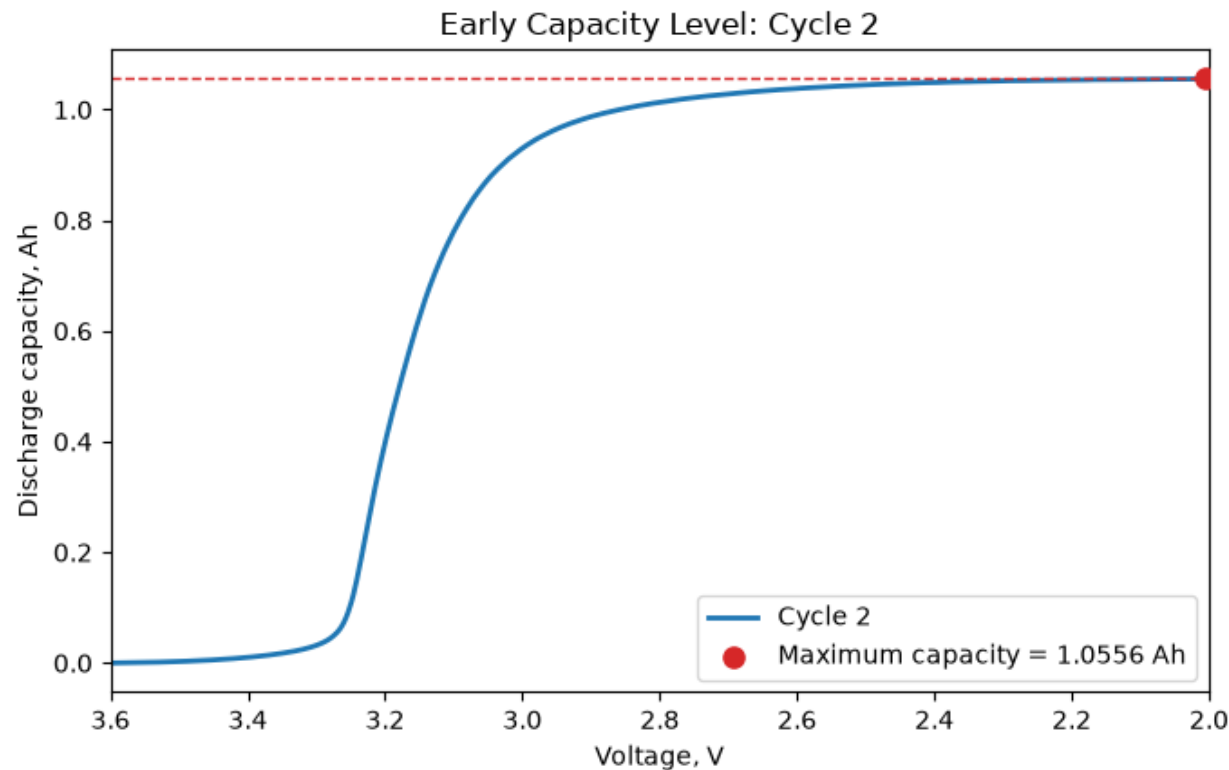


Features

$$\Delta Q(V) = Q_{100}(V) - Q_{10}(V)$$

Where $Q_n(V)$ is the discharge capacity at voltage V during cycle n . Each curve contains 1000 voltage points.

Features	Meaning
max_Qdlin_cycle2_Ah	Maximum value of the discharge-capacity curve at cycle 2. Describes the battery's early capacity level.

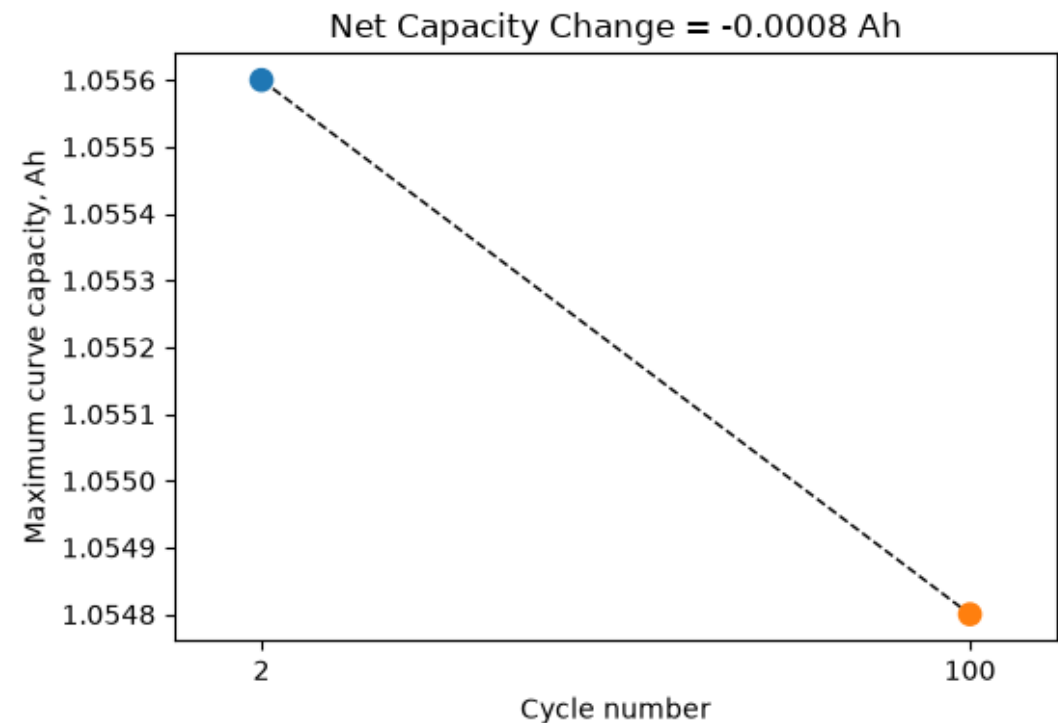
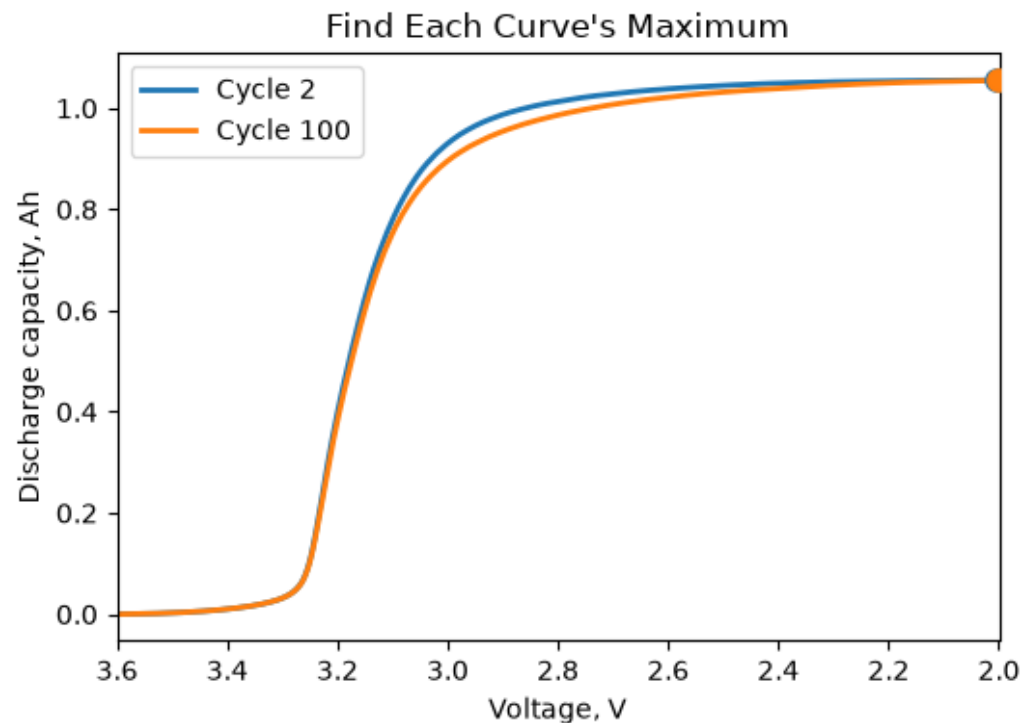


Features

$$\Delta Q(V) = Q_{100}(V) - Q_{10}(V)$$

Where $Q_n(V)$ is the discharge capacity at voltage V during cycle n . Each curve contains 1000 voltage points.

Features	Meaning
max_Qdlin_cycle100_minus2_Ah	Maximum curve capacity at cycle 100 minus maximum curve capacity at cycle 2. Measures the net change in that capacity measure, in Ah.

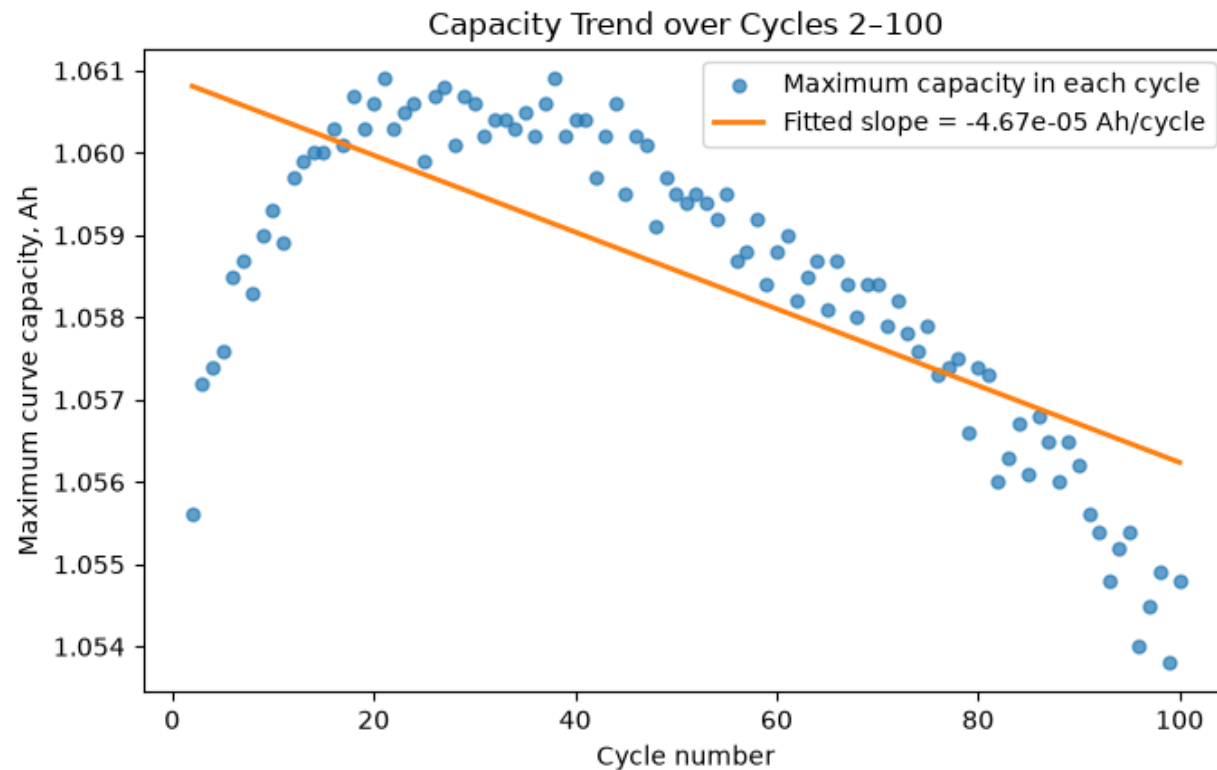


Features

$$\Delta Q(V) = Q_{100}(V) - Q_{10}(V)$$

Where $Q_n(V)$ is the discharge capacity at voltage V during cycle n . Each curve contains 1000 voltage points.

Features	Meaning
max_Qdlin_slope_cycles2_100_Ah_per_cycle	Slope of a straight line fitted to the maximum curve capacity versus cycle number over cycles 2–100. Measures its average trend, in Ah/cycle.

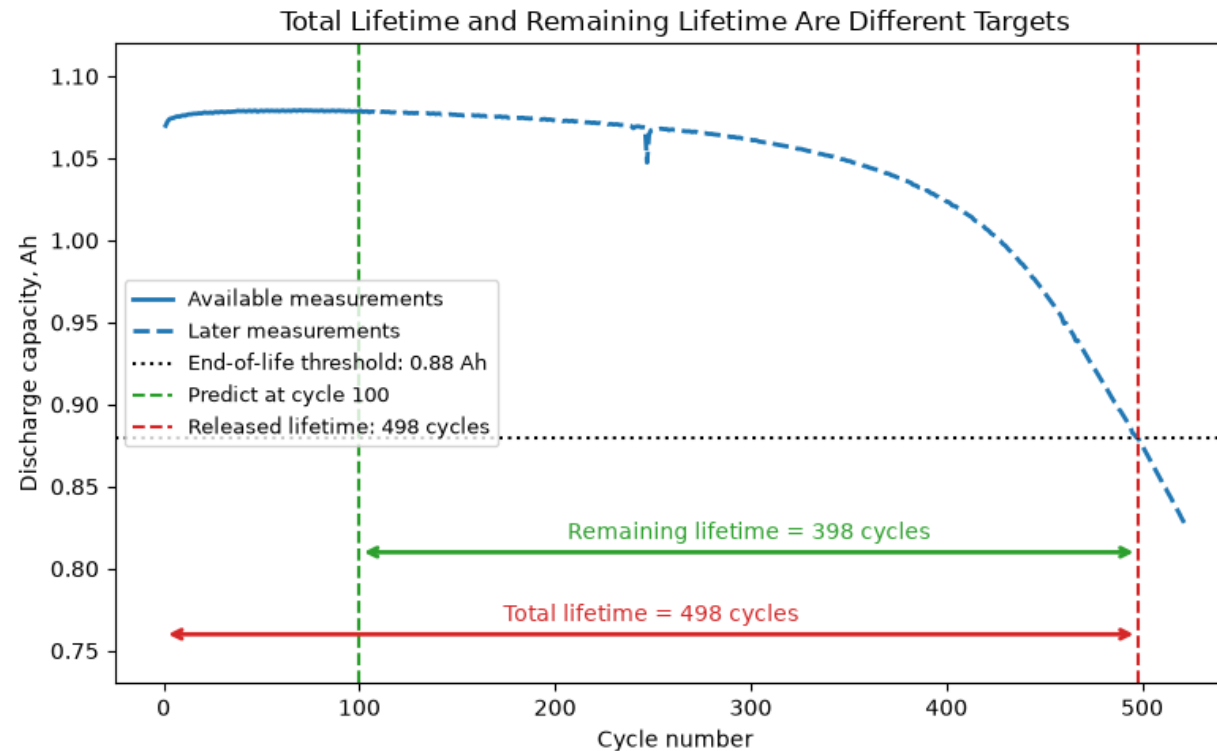


Targets

$$\Delta Q(V) = Q_{100}(V) - Q_{10}(V)$$

Where $Q_n(V)$ is the discharge capacity at voltage V during cycle n . Each curve contains 1000 voltage points.

Targets	Meaning
cycle_life	Total battery lifetime, in cycles, using the released 80%-capacity lifetime label. It is not the number of cycles remaining after cycle 100.



Preparing Battery Data for Ridge Regression

```
import numpy as np
import pandas as pd

from sklearn.linear_model import Ridge
from sklearn.metrics import root_mean_squared_error
from sklearn.preprocessing import StandardScaler

# 1. Read the data and define X and y.
data = pd.read_csv("data/battery/features.csv")

feature_columns = [
    "log10_var_dQ100_10",
    "log10_abs_min_dQ100_10",
    "mean_dQ100_10_Ah",
    "max_Qdlin_cycle2_Ah",
    "max_Qdlin_cycle100_minus2_Ah",
    "max_Qdlin_slope_cycles2_100_Ah_per_cycle",
]

X = data[feature_columns].to_numpy()
y = data["cycle_life"].to_numpy()

print("Number of data points:", X.shape[0])
print("Number of features:", X.shape[1])
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
Number of data points: 124
Number of features: 6
X shape: (124, 6)
y shape: (124,)
```

```
# 2. Use the primary test set as validation for this exercise.
train = data["split"].eq("train").to_numpy()
val = data["split"].eq("primary_test").to_numpy()
test = data["split"].eq("secondary_test").to_numpy()

X_train, y_train = X[train], y[train]
X_val, y_val = X[val], y[val]
X_test, y_test = X[test], y[test]

print("\nTraining:", X_train.shape, y_train.shape)
print("Validation:", X_val.shape, y_val.shape)
print("Test:", X_test.shape, y_test.shape)
```

Select indices

```
# 3. Fit the scaler on training data only.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
```

Selecting Ridge Strength Using Validation Error

4. Select alpha using validation RMSE.

```
alpha_values = np.logspace(-4, 4, 41)
```

```
validation_rmse = []
```

```
for alpha in alpha_values:
    model = Ridge(alpha=alpha)
    model.fit(X_train_scaled, y_train)
    y_val_pred = model.predict(X_val_scaled)
    validation_rmse.append(root_mean_squared_error(y_val, y_val_pred))
```

```
best_index = np.argmin(validation_rmse)
```

```
best_alpha = alpha_values[best_index]
```

```
print(f"\nSelected alpha: {best_alpha:.4g}")
```

```
print(f"Validation RMSE: {validation_rmse[best_index]:.1f} cycles")
```

Selected alpha: 0.0001

Validation RMSE: 153.1 cycles

5. Refit the scaler and selected model on training + validation.

```
X_train_val = np.vstack([X_train, X_val])
```

```
y_train_val = np.concatenate([y_train, y_val])
```

```
final_scaler = StandardScaler()
```

```
X_train_val_scaled = final_scaler.fit_transform(X_train_val)
```

```
X_test_scaled = final_scaler.transform(X_test)
```

```
final_model = Ridge(alpha=best_alpha)
```

```
final_model.fit(X_train_val_scaled, y_train_val)
```

6. Evaluate once on the untouched test set.

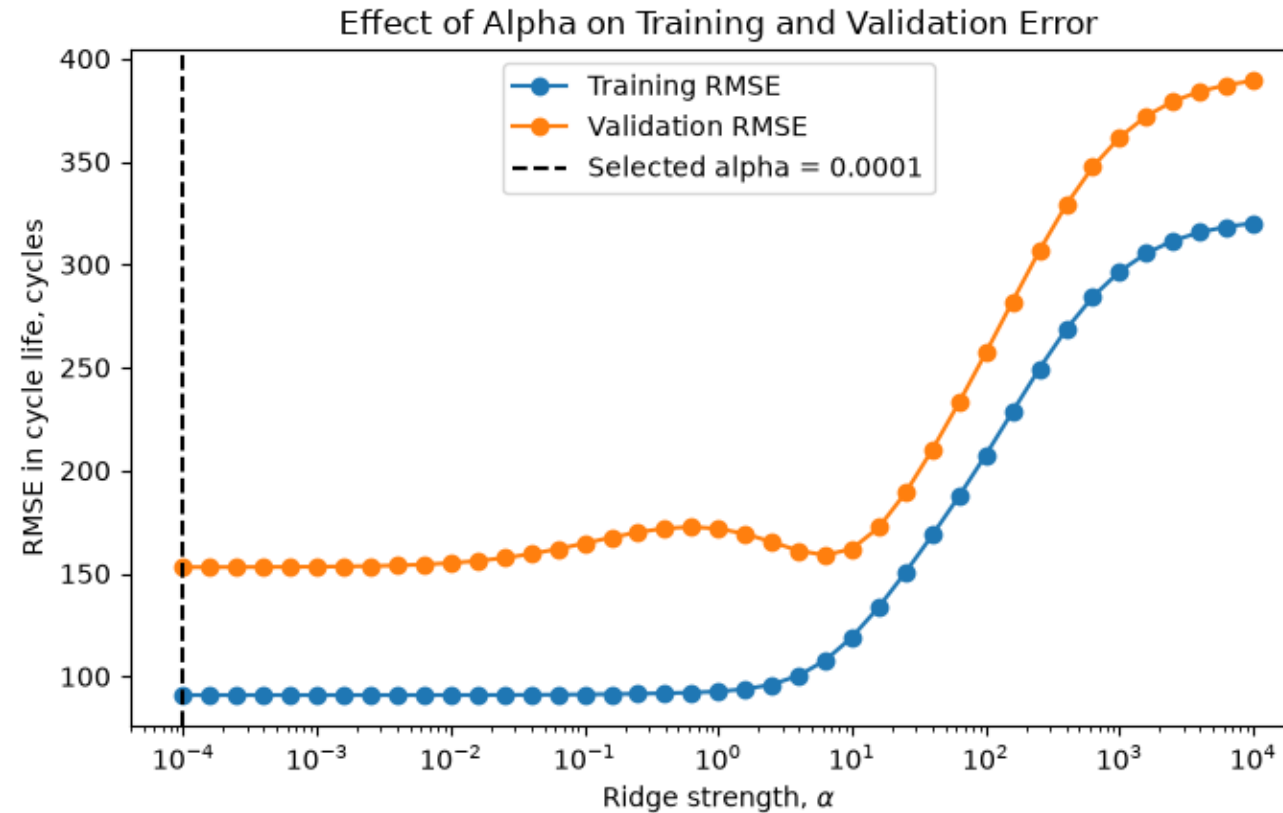
```
y_test_pred = final_model.predict(X_test_scaled)
```

```
test_rmse = root_mean_squared_error(y_test, y_test_pred)
```

```
print(f"Final test RMSE: {test_rmse:.1f} cycles")
```

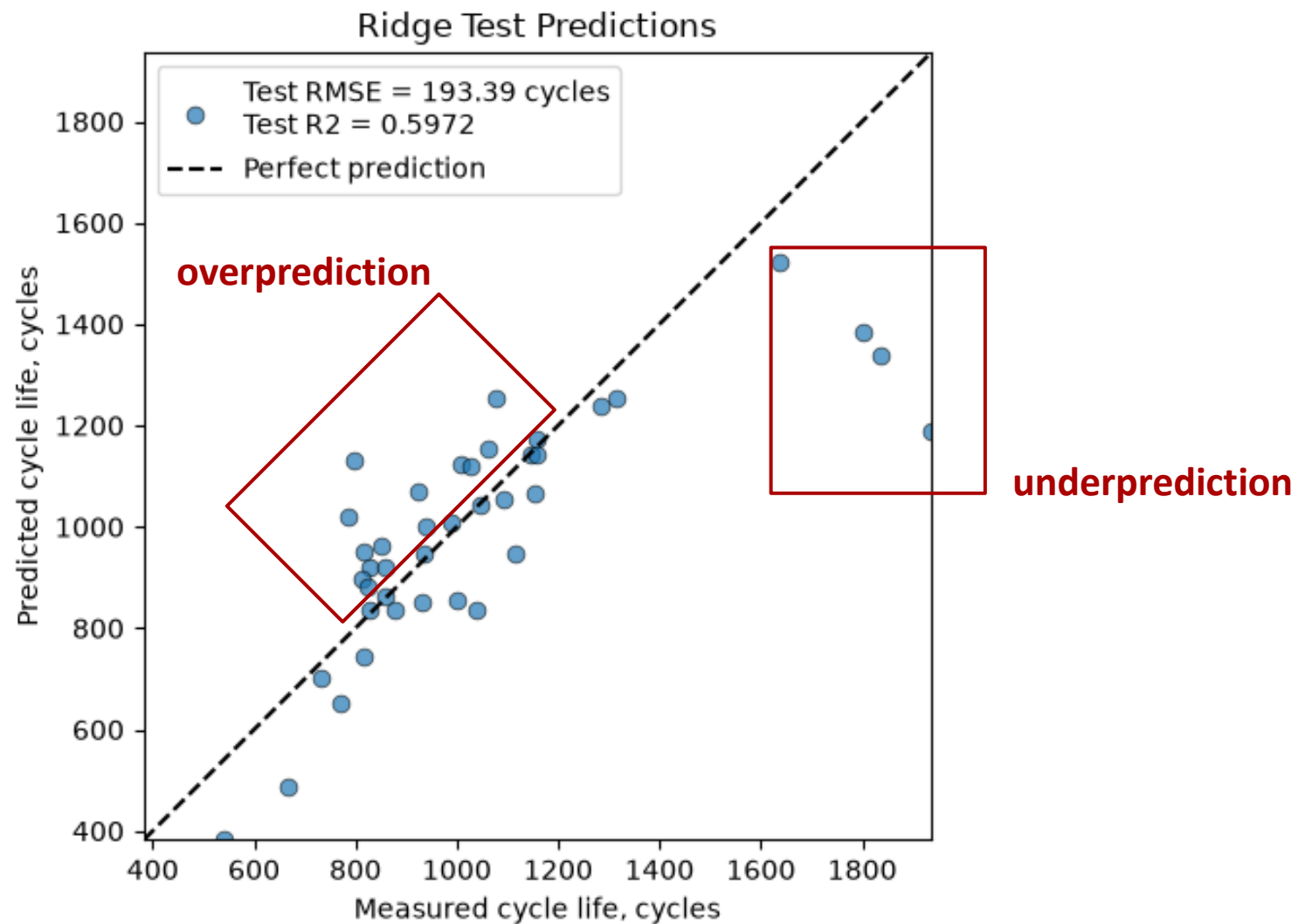
Final test RMSE: 193.4 cycles

The Impact of Alpha on Prediction Error



Small alpha values produce nearly identical validation errors. As alpha increases, stronger coefficient shrinkage eventually raises both training and validation errors, indicating underfitting.

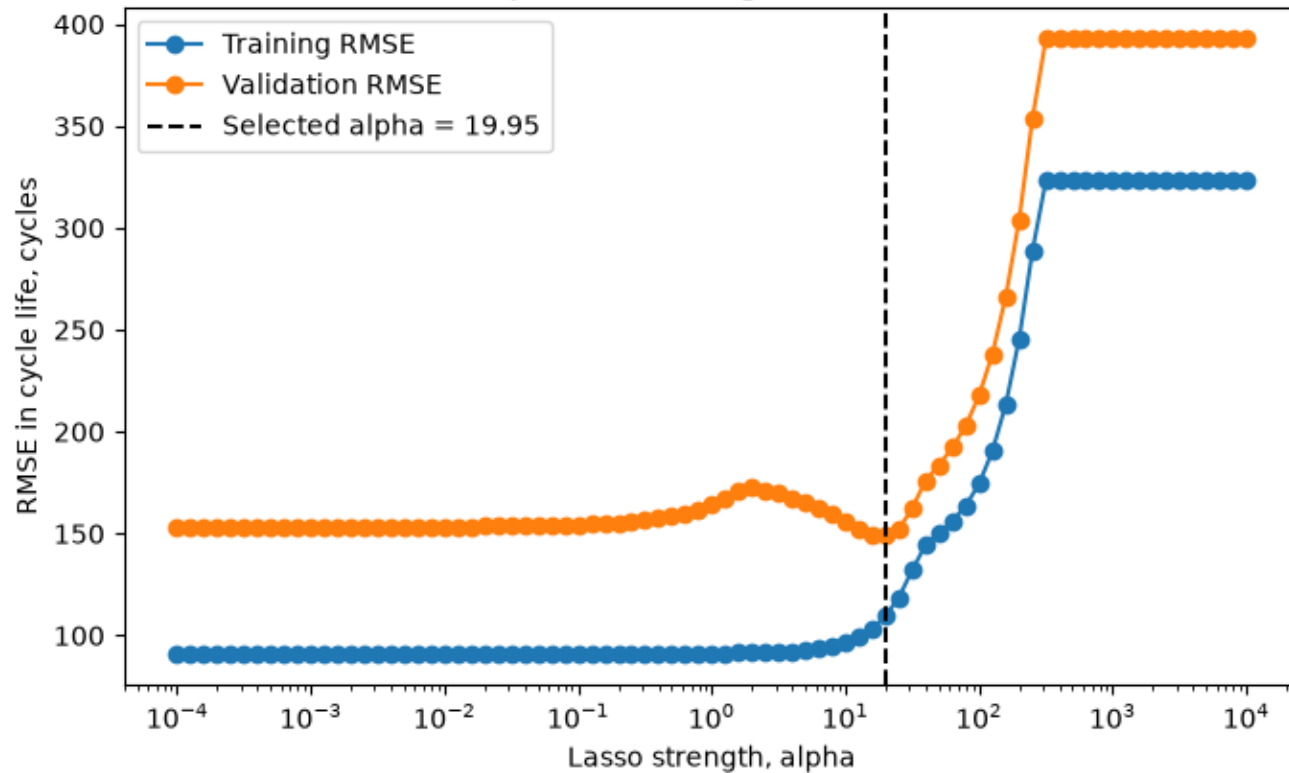
Ridge Predictions on the Final Test Set



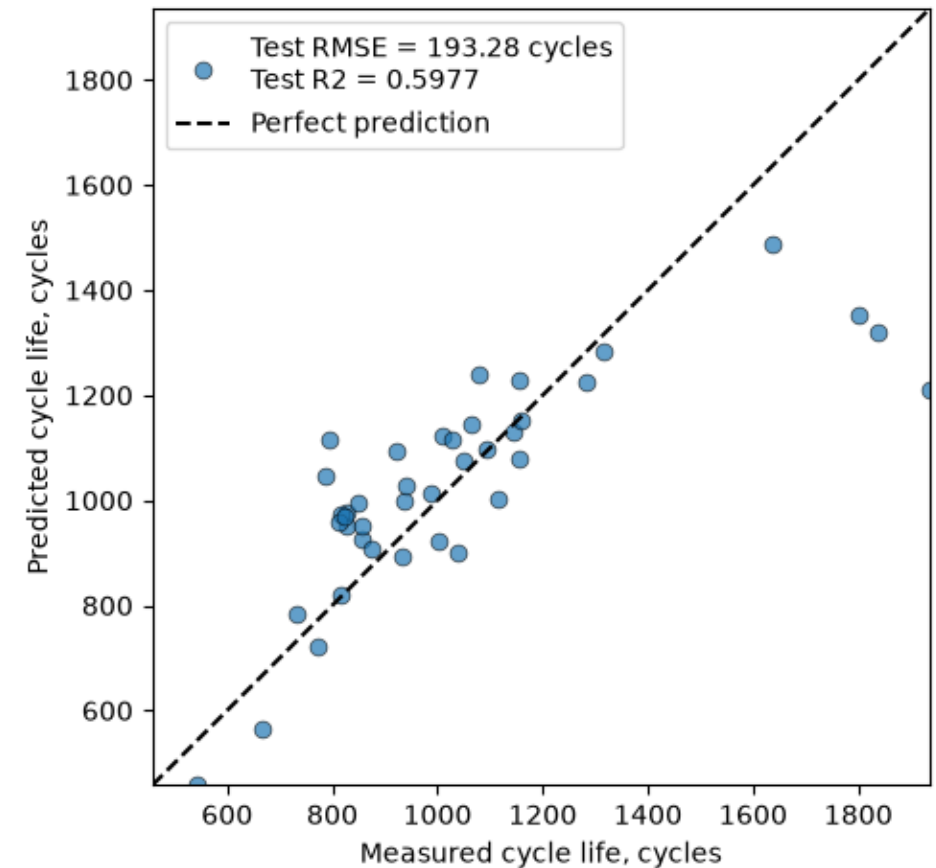
Each point represents one test battery. Perfect predictions lie on the diagonal; points below it indicate **underprediction**. The model tends to underestimate the lifetimes of the longest-lived batteries.

The Impact of Lasso on Prediction Performance

Effect of Alpha on Training and Validation Error



Lasso Test Predictions



Lasso can shrink some coefficients exactly to zero.
Here, its test performance is essentially the same as Ridge.

K-Fold Cross-Validation for Model Selection

A primary goal of many machine learning regression problems is to achieve models that permit **generalization** (evaluation outside the domain of our dataset)

Cross-validation and/or the use of **train/validation/test splits** are two common strategies to mitigate overfitting a model to example data/inhibiting generalization

cross validation provides a more structured way to guide model selection

k-fold Cross-Validation

1. Shuffle dataset *randomly*
2. Set aside the **test set**
3. Split the remaining data into k folds
4. For each unique group
 - i. Hold out the group for evaluation as validation data
 - ii. Assign remaining groups as training data
 - iii. Optimize a model on the training data
 - iv. Evaluate model performance
 - v. Store model parameters and evaluation
5. Choose the hyperparameter using the average validation error
6. Refit on all non-test data using the selected hyperparameter, then evaluate once on the test set

$\mathcal{E}_1, \theta_1$

$\vdots$

$\mathcal{E}_5, \theta_5$

Shuffled Data				
Fold 1	Fold 2	Fold 3	Fold 4	Fold 5

Choosing Cross Validation Methods for Different Data

A primary goal of many machine learning regression problems is to achieve models that permit **generalization** (evaluation outside the domain of our dataset)

Cross-validation and/or the use of **train/validation/test splits** are two common strategies to mitigate overfitting a model to example data/inhibiting generalization

Some Flavors of Cross-Validation

K-fold: each fold takes a turn as validation; 5 or 10 folds are common.

Leave-one-out: validate on one sample at a time and train on all remaining samples.

Stratified k-fold: preserve class proportions across folds for classification.

Repeated k-fold: repeat with different random splits to examine sensitivity to the split.

Common Cross-Validation Methods

sklearn.model_selection: Model Selection

User guide: See the [Cross-validation: evaluating estimator performance](#), [Tuning the hyper-parameters of an estimator](#) and [Learning curve](#) sections for further details.

Splitter Classes

<code>model_selection.GroupKFold([n_splits])</code>	K-fold iterator variant with non-overlapping groups.
<code>model_selection.GroupShuffleSplit([...])</code>	Shuffle-Group(s)-Out cross-validation iterator
<code>model_selection.KFold([n_splits, shuffle, ...])</code>	K-Folds cross-validator
<code>model_selection.LeaveOneGroupOut()</code>	Leave One Group Out cross-validator
<code>model_selection.LeavePGroupsOut(n_groups)</code>	Leave P Group(s) Out cross-validator
<code>model_selection.LeaveOneOut()</code>	Leave-One-Out cross-validator
<code>model_selection.LeavePOut(p)</code>	Leave-P-Out cross-validator
<code>model_selection.PredefinedSplit(test_fold)</code>	Predefined split cross-validator
<code>model_selection.RepeatedKFold(*[, n_splits, ...])</code>	Repeated K-Fold cross validator.
<code>model_selection.RepeatedStratifiedKFold(*[, ...])</code>	Repeated Stratified K-Fold cross validator.
<code>model_selection.ShuffleSplit([n_splits, ...])</code>	Random permutation cross-validator
<code>model_selection.StratifiedKFold([n_splits, ...])</code>	Stratified K-Folds cross-validator.
<code>model_selection.StratifiedShuffleSplit([...])</code>	Stratified ShuffleSplit cross-validator
<code>model_selection.StratifiedGroupKFold([...])</code>	Stratified K-Folds iterator variant with non-overlapping groups.
<code>model_selection.TimeSeriesSplit([n_splits, ...])</code>	Time Series cross-validator

sklearn.model_selection.KFold

```
class sklearn.model_selection.KFold(n_splits=5, *, shuffle=False, random_state=None)
```

[\[source\]](#)

K-Folds cross-validator

Provides train/test indices to split data in train/test sets. Split dataset into k consecutive folds (without shuffling by default).

Each fold is then used once as a validation while the k - 1 remaining folds form the training set.

Read more in the [User Guide](#).

Parameters:

n_splits : int, default=5

Number of folds. Must be at least 2.

Changed in version 0.22: `n_splits` default value changed from 3 to 5.

shuffle : bool, default=False

Whether to shuffle the data before splitting into batches. Note that the samples within each split will not be shuffled.

random_state : int, RandomState instance or None, default=None

When `shuffle` is True, `random_state` affects the ordering of the indices, which controls the randomness of each fold. Otherwise, this parameter has no effect. Pass an int for reproducible output across multiple function calls. See [Glossary](#).

Manual Cross-Validation Loop

1. Load the data and reserve the final test set

```
from sklearn.linear_model import Lasso
from sklearn.metrics import root_mean_squared_error, r2_score
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler

data = pd.read_csv("data/battery/features.csv")

feature_columns = [
    "log10_var_dQ100_10",
    "log10_abs_min_dQ100_10",
    "mean_dQ100_10_Ah",
    "max_Qdlin_cycle2_Ah",
    "max_Qdlin_cycle100_minus2_Ah",
    "max_Qdlin_slope_cycles2_100_Ah_per_cycle",
]

X = data[feature_columns].to_numpy()
y = data["cycle_life"].to_numpy()
```

```
# Combine the previous training and validation sets for cross-validation.
development = data["split"].isin(["train", "primary_test"]).to_numpy()
test = data["split"].eq("secondary_test").to_numpy()
```

```
X_dev, y_dev = X[development], y[development]
X_test, y_test = X[test], y[test]
```

```
print("Total data points:", X.shape[0])
print("Number of features:", X.shape[1])
print("Development shapes:", X_dev.shape, y_dev.shape)
print("Test shapes:", X_test.shape, y_test.shape)
```

```
Total data points: 124
Number of features: 6
Development shapes: (84, 6) (84,)
Test shapes: (40, 6) (40,)
```

Manual Cross-Validation Loop

2. Create and visualize the five folds

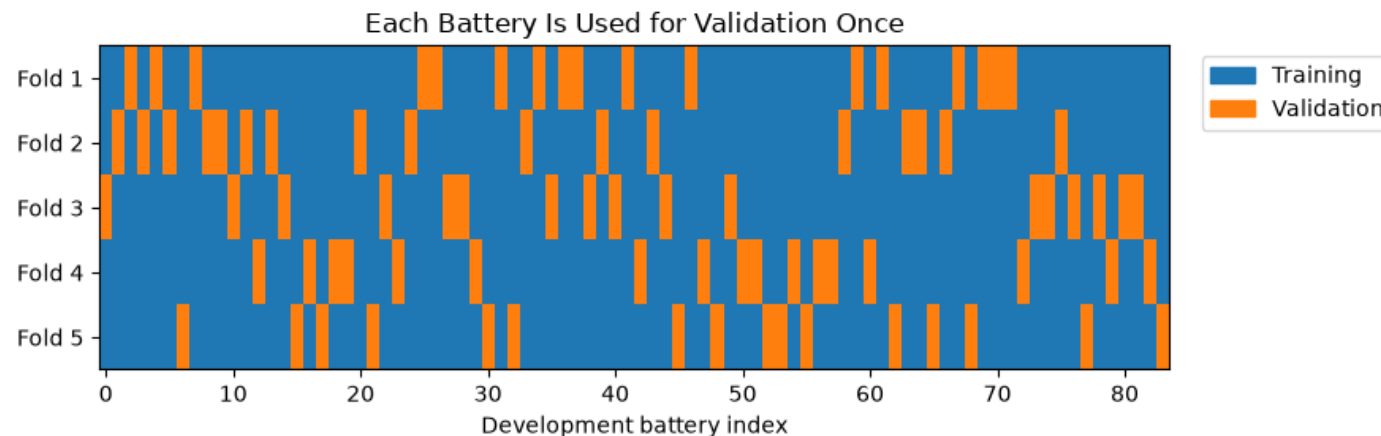
```
from matplotlib.colors import ListedColormap
from matplotlib.patches import Patch

cv = KFold(n_splits=5, shuffle=True, random_state=26)
folds = list(cv.split(X_dev))

fold_map = np.zeros((5, len(y_dev)))

for fold, (fit_index, val_index) in enumerate(folds):
    fold_map[fold, val_index] = 1
    print(f"Fold {fold + 1}: {len(fit_index)} training, {len(val_index)} validation")
```

Fold 1: 67 training, 17 validation
Fold 2: 67 training, 17 validation
Fold 3: 67 training, 17 validation
Fold 4: 67 training, 17 validation
Fold 5: 68 training, 16 validation



Manual Cross-Validation Loop

3. Demonstrate cross-validation for one alpha

```
alpha = 1.0
fold_results = []

for fold, (fit_index, val_index) in enumerate(folds, start=1):
    X_fit, y_fit = X_dev[fit_index], y_dev[fit_index]
    X_val, y_val = X_dev[val_index], y_dev[val_index]

    # Fit a new scaler using only this fold's fitting batteries.
    scaler = StandardScaler()
    X_fit_scaled = scaler.fit_transform(X_fit)
    X_val_scaled = scaler.transform(X_val)

    model = Lasso(alpha=alpha, max_iter=100000)
    model.fit(X_fit_scaled, y_fit)

    train_error = root_mean_squared_error(y_fit, model.predict(X_fit_scaled))
    val_error = root_mean_squared_error(y_val, model.predict(X_val_scaled))

    fold_results.append({"Fold": fold, "Training RMSE": train_error,
                        "Validation RMSE": val_error})

fold_results = pd.DataFrame(fold_results)
display(fold_results.round(2))

print(f"Alpha: {alpha}")
print(f"Mean validation RMSE: {fold_results['Validation RMSE'].mean():.2f} cycles")
print(f"Validation RMSE standard deviation: {fold_results['Validation RMSE'].std():.2f} cyc
```

	Fold	Training RMSE	Validation RMSE
0	1	97.14	120.62
1	2	103.17	78.45
2	3	96.20	111.75
3	4	89.06	162.50
4	5	103.70	185.64

Alpha: 1.0

Mean validation RMSE: 131.79 cycles

Validation RMSE standard deviation: 42.47 cycles

Manual Cross-Validation Loop

4. Compare alpha values using the same five folds

```
alpha_values = np.logspace(-4, 4, 81)

train_rmse = np.zeros((len(alpha_values), len(folds)))
validation_rmse = np.zeros((len(alpha_values), len(folds)))

for i, alpha in enumerate(alpha_values):
    for j, (fit_index, val_index) in enumerate(folds):
        X_fit, y_fit = X_dev[fit_index], y_dev[fit_index]
        X_val, y_val = X_dev[val_index], y_dev[val_index]

        scaler = StandardScaler()
        X_fit_scaled = scaler.fit_transform(X_fit)
        X_val_scaled = scaler.transform(X_val)

        model = Lasso(alpha=alpha, max_iter=100000)
        model.fit(X_fit_scaled, y_fit)

        train_rmse[i, j] = root_mean_squared_error(y_fit, model.predict(X_fit_scaled))
        validation_rmse[i, j] = root_mean_squared_error(y_val, model.predict(X_val_scaled))

mean_train_rmse = train_rmse.mean(axis=1)
mean_validation_rmse = validation_rmse.mean(axis=1)
std_validation_rmse = validation_rmse.std(axis=1, ddof=1)

best_index = np.argmin(mean_validation_rmse)
best_alpha = alpha_values[best_index]

print(f"Selected alpha: {best_alpha:.4g}")
print(f"Mean validation RMSE: {mean_validation_rmse[best_index]:.2f} cycles")
```

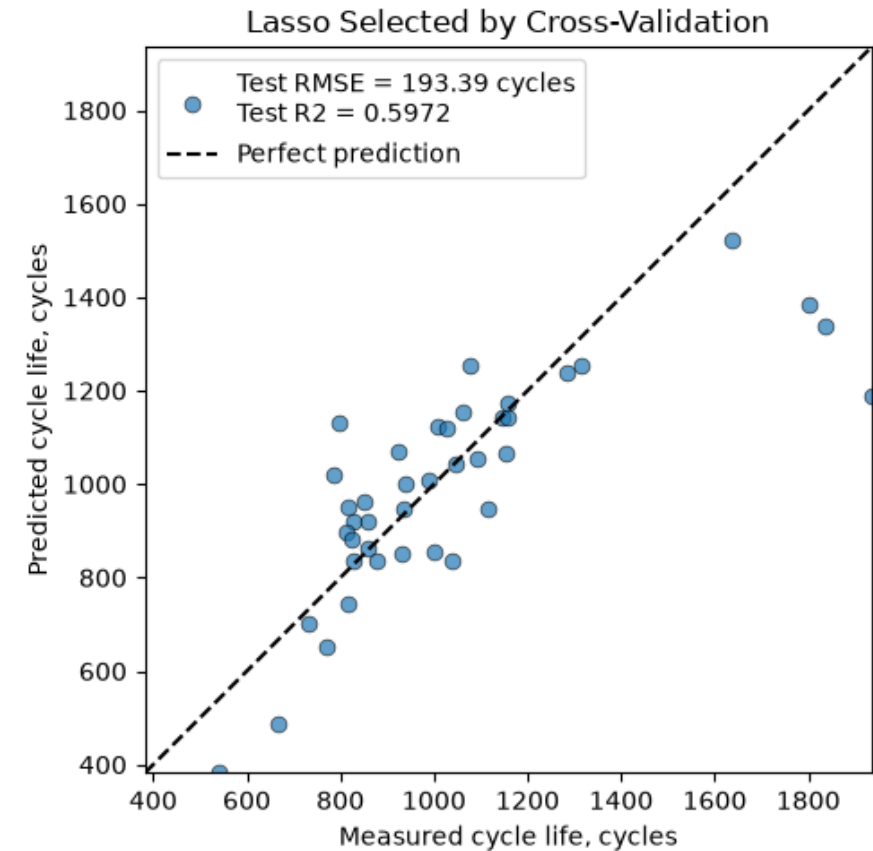
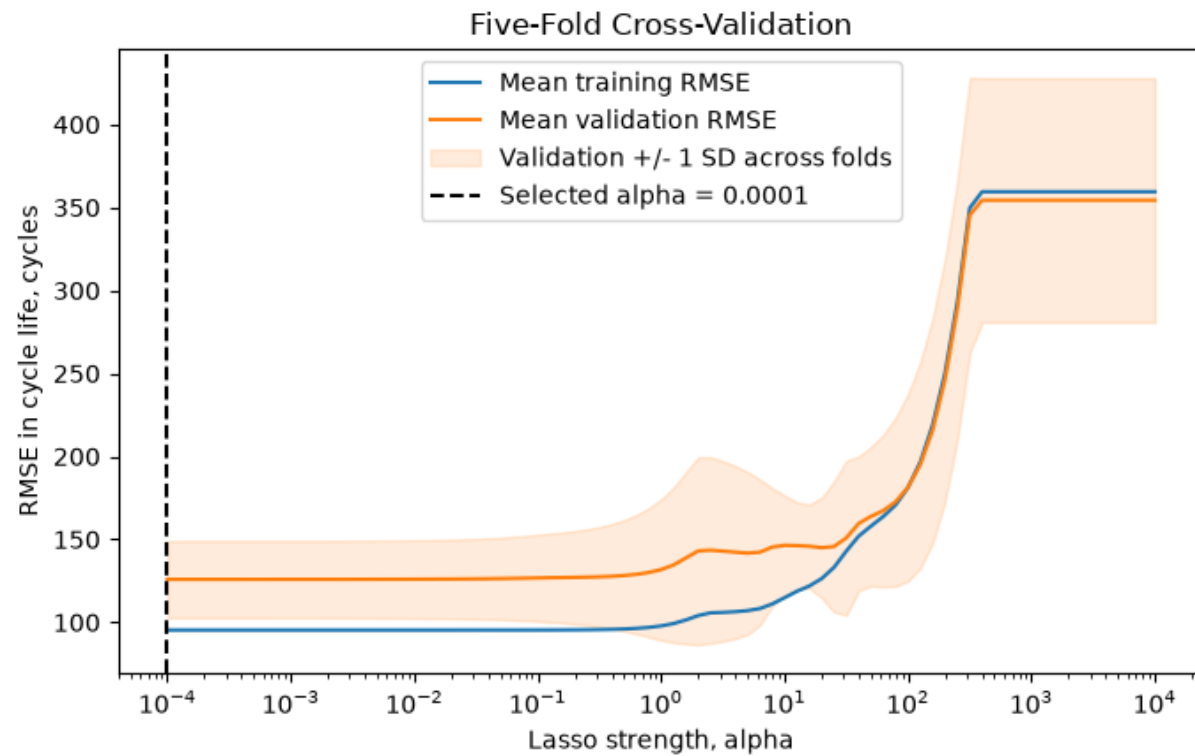
```
Selected alpha: 0.0001
Mean validation RMSE: 125.92 cycles
```

Manual Cross-Validation Loop

6. Refit on all 84 development batteries

```
final_scaler = StandardScaler()
X_dev_scaled = final_scaler.fit_transform(X_dev)
X_test_scaled = final_scaler.transform(X_test)

final_model = Lasso(alpha=best_alpha, max_iter=100000)
final_model.fit(X_dev_scaled, y_dev)
```

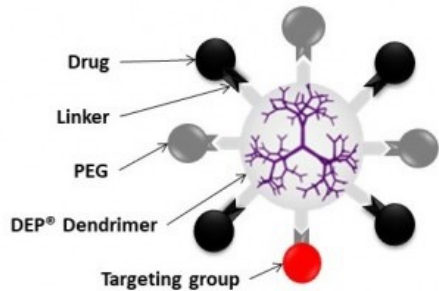


Cross-validation does not guarantee improved performance; it is a tool for evaluating model generalization and guiding model selection.

Stratified Cross-validation: A Polymer Architecture Example



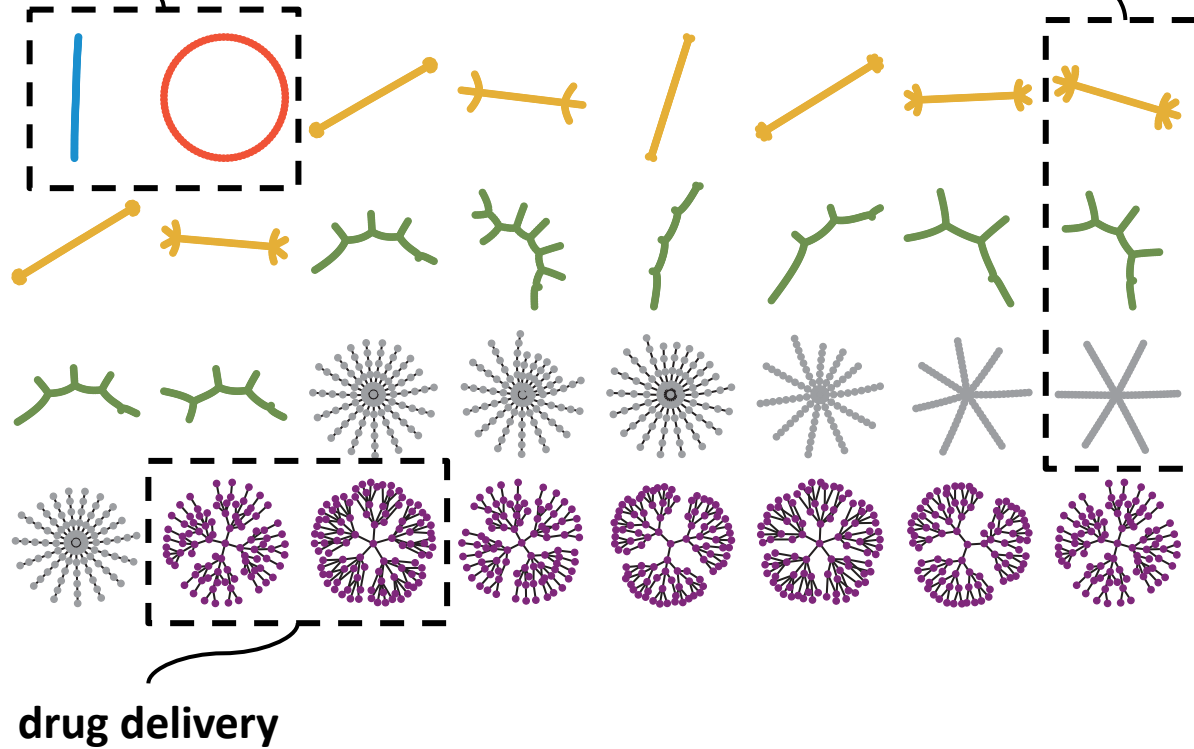
supercool.com.au



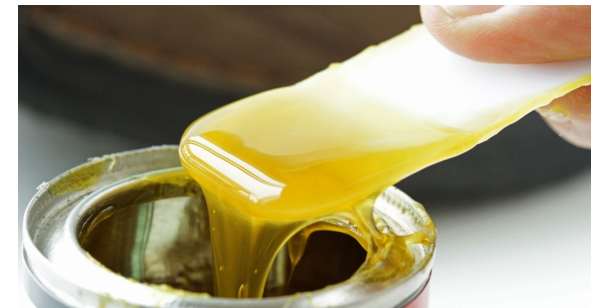
starpharma.com

viscosity, lubricant

surface coating,
adhesives



dimarmfg.com

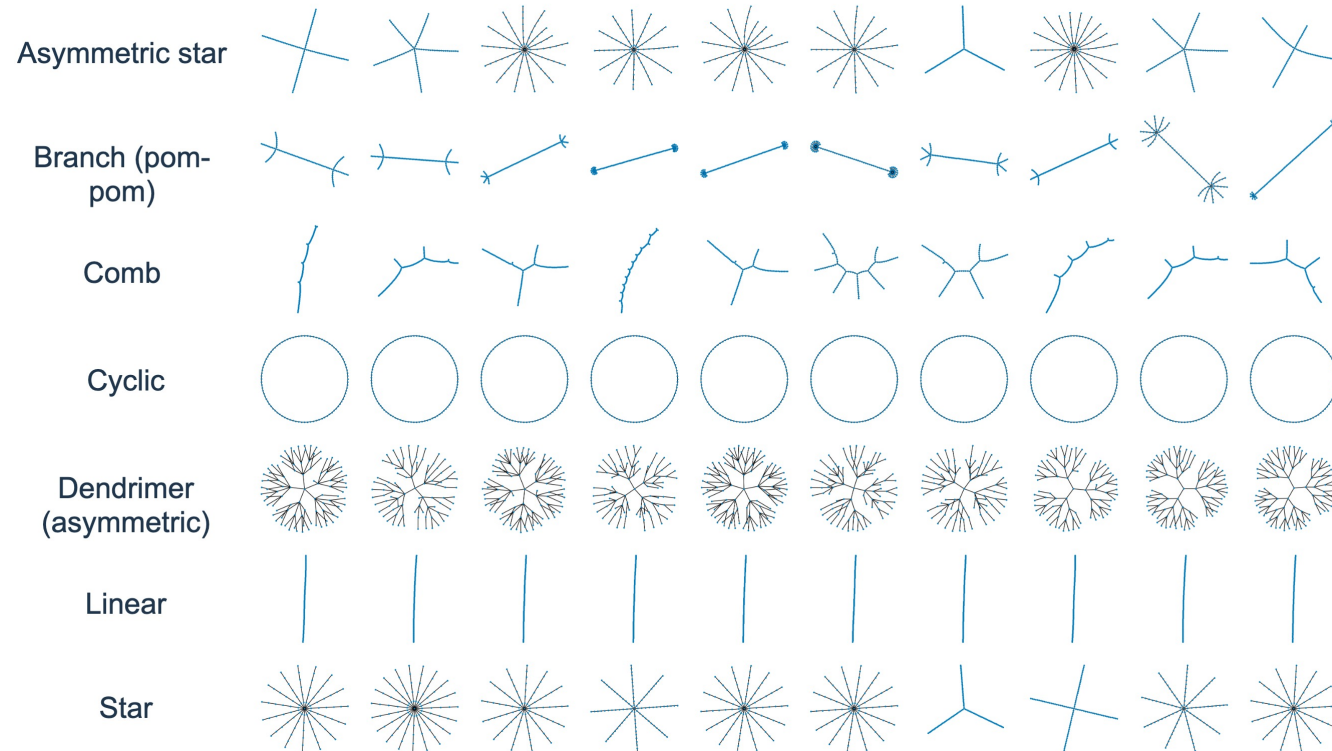


superbond.co.in

Machine learning has primarily been used to study **linear polymers**.

How Polymer Architecture Imbalance Affects Validation

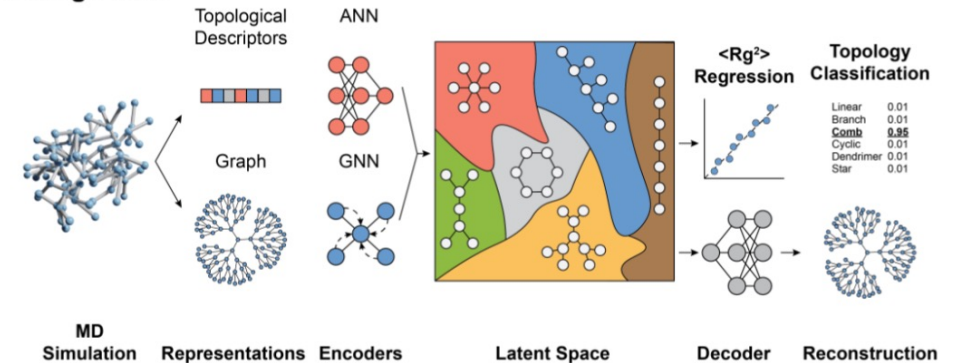
polymer architectures with mass constraints



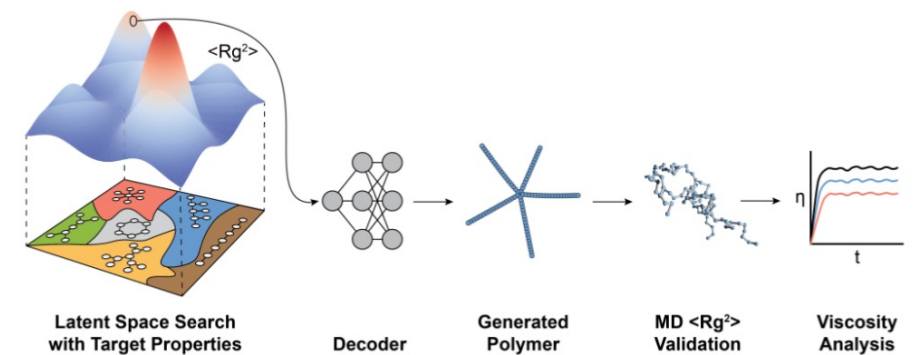
Topology	Number
Linear	11
Cyclic	11
Star/asymmetric star	24/306
Branch (pom-pom)	330
Comb	330
Dendrimer	330

significant class imbalance

I. Training Phase



II. Search Phase



- Some polymer architectures are much rarer than others. **Stratifying** by architecture keeps their proportions similar across folds, while the regression target remains mean squared radius of gyration.

Stratification Preserves Battery Type Proportions Across Folds

```
import numpy as np
from sklearn.model_selection import KFold, StratifiedKFold

# Imbalanced toy dataset: 12 LFP, 6 LCO, and 3 Li-S batteries.
battery_type = np.array(["LFP"] * 12 + ["LCO"] * 6 + ["Li-S"] * 3)
X = np.arange(21).reshape(-1, 1)

methods = {
    "KFold": KFold(n_splits=3, shuffle=True, random_state=26),
    "StratifiedKFold": StratifiedKFold(n_splits=3, shuffle=True, random_state=26),
}

for name, cv in methods.items():
    print(f"\n{name}")
    for fold, (_, val) in enumerate(cv.split(X, battery_type), start=1):
        types, counts = np.unique(battery_type[val], return_counts=True)
        print(f"Fold {fold}: {dict(zip(types.tolist(), counts.tolist()))}")
```

KFold

Fold 1: {'LCO': 4, 'LFP': 2, 'Li-S': 1}

Fold 2: {'LFP': 6, 'Li-S': 1}

Fold 3: {'LCO': 2, 'LFP': 4, 'Li-S': 1}

StratifiedKFold

Fold 1: {'LCO': 2, 'LFP': 4, 'Li-S': 1}

Fold 2: {'LCO': 2, 'LFP': 4, 'Li-S': 1}

Fold 3: {'LCO': 2, 'LFP': 4, 'Li-S': 1}

Automating Lasso Tuning with Grid Search

```
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.pipeline import Pipeline
from sklearn.metrics import root_mean_squared_error, r2_score

lasso_model = Pipeline([("scaler", StandardScaler()), ("lasso", Lasso(max_iter=100000))])
lasso_parameters = {"lasso__alpha": np.logspace(-4, 4, 81)}

cv = KFold(n_splits=5, shuffle=True, random_state=26)
lasso_search = GridSearchCV(lasso_model, lasso_parameters, cv=cv, scoring="neg_root_mean_squared_error")
lasso_search.fit(X_dev, y_dev)

lasso_test_pred = lasso_search.predict(X_test)

print("Lasso best alpha:", lasso_search.best_params_["lasso__alpha"])
print(f"Lasso mean validation RMSE: {-lasso_search.best_score_: .2f} cycles")
print(f"Lasso test RMSE: {root_mean_squared_error(y_test, lasso_test_pred): .2f} cycles")
print(f"Lasso test R2: {r2_score(y_test, lasso_test_pred): .4f}")

Lasso best alpha: 0.0001
Lasso mean validation RMSE: 125.92 cycles
Lasso test RMSE: 193.39 cycles
Lasso test R2: 0.5972
```

GridSearchCV evaluates 81 alpha values using five folds: 405 validation fits. It selects the lowest mean RMSE and performs one final refit. The pipeline keeps scaling inside each fold.

Joint Tuning of Elastic Net Hyperparameters

```
elastic_model = Pipeline([("scaler", StandardScaler()), ("elastic", ElasticNet(max_iter=100000))])
elastic_parameters = {
    "elastic__alpha": np.logspace(-4, 4, 41),
    "elastic__l1_ratio": [0.1, 0.3, 0.5, 0.7, 0.9, 1.0],
}

elastic_search = GridSearchCV(elastic_model, elastic_parameters, cv=cv, scoring="neg_root_mean_squared_error")
elastic_search.fit(X_dev, y_dev)

elastic_test_pred = elastic_search.predict(X_test)

print("Elastic Net best alpha:", elastic_search.best_params_["elastic__alpha"])
print("Elastic Net best l1_ratio:", elastic_search.best_params_["elastic__l1_ratio"])
print(f"Elastic Net mean validation RMSE: {-elastic_search.best_score_: .2f} cycles")
print(f"Elastic Net test RMSE: {root_mean_squared_error(y_test, elastic_test_pred): .2f} cycles")
print(f"Elastic Net test R2: {r2_score(y_test, elastic_test_pred): .4f}")
```

```
Elastic Net best alpha: 0.0001
Elastic Net best l1_ratio: 1.0
Elastic Net mean validation RMSE: 125.92 cycles
Elastic Net test RMSE: 193.39 cycles
Elastic Net test R2: 0.5972
```

We search 246 combinations. The selected l1_ratio of 1.0 uses only the L1 penalty, giving the Lasso solution.

Interactive Python Time

09_hyperparameter_tuning.ipynb